Transformers-compatible model checkpoint from Hugging Face. Preserve the exact model identifier and revision; an adapter is tied to the base architecture and weights it was trained against.

Run your first LoRA experiment

Step 1: Create the files. Place the reviewed prompt-completion records in `data/training/train.jsonl` and `data/training/validation.jsonl`. Make sure each line is valid JSON and both files share the same structure. Leave test questions under `data/evaluation`.

Step 2: Create `src/train_lora.py`. Enter the following compact example, which uses the current Hugging Face TRL and PEFT interfaces:

```
from datasets import load_dataset
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer

data = load_dataset(
    "json",
    data_files={
        "train": "data/training/train.jsonl",
        "validation": "data/training/validation.jsonl"
    }
)

args = SFTConfig(
    output_dir="outputs/support-lora",
    num_train_epochs=2,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=1e-4,
    max_length=512,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=5,
    completion_only_loss=True,
    report_to="none"
)

adapter = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    target_modules="all-linear",
    task_type="CAUSAL_LM"
)

trainer = SFTTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    args=args,
    train_dataset=data["train"],
    eval_dataset=data["validation"],
    peft_config=adapter
)

trainer.train()
trainer.save_model("models/support-lora")
```

Optional preflight: Before committing to two epochs, test the environment with a short run on a handful of synthetic, non-sensitive examples. Check that the tokenizer accepts your schema, that completions are not cut off at 512 tokens and that at least one optimiser step completes. If the GPU is already near capacity before training begins, reduce memory demands rather than relying on luck.

Step 3: Start training. Run `python src/train_lora.py` from the project root. The first execution may download the base checkpoint. Watch the output rather than closing the terminal. A memory error is a signal to reduce sequence length or batch size, not to assume the model is defective.

Step 4: Inspect what was saved. Under `models/support-lora`, look for adapter configuration and adapter weights. These are not a standalone replacement for the base model. Save the tokenizer or its exact identifier alongside your experiment notes so inference uses the same formatting.

Step 5: Compare fairly. Ask the unchanged base model and the adapted model the same held-out questions using identical instructions and generation settings. Count valid JSON outputs, correct categories, appropriate urgency and unsupported actions. With only a few dozen examples, expect an educational demonstration rather than a miraculous transformation.

Diagnose the first failure

If the trainer cannot read your JSONL file, inspect a single line with Python's `json.loads()` and confirm the `prompt` and `completion` keys match the documented format. If you hit an out-of-memory error, first reduce `max_length` and batch size, then consider QLoRA. If the job runs but loss becomes `nan`, stop and check numeric precision, learning rate and malformed examples. If the adapter saves yet generation looks unchanged, verify that you actually loaded it on the correct base checkpoint. A common mistake is benchmarking the original Ollama model while the new adapter sits unused in another directory. Record the fix; tomorrow's experiment should not rediscover today's problem.

Understand the settings that matter

Training controls look intimidating until you connect them to familiar trade-offs. Learning rate determines the size of optimisation updates. Too high may destabilise learning; too low may